Algorithmes et Pensée Computationnelle

Consolidation POO + Classes abstraites et interfaces

Les exercices sont construits autour des concepts d'héritage, de classes abstraites et d'interfaces. Au terme de cette séance, vous serez en mesure de différencier une classe abstraite d'une interface, de savoir à quel moment utiliser l'un ou l'autre et de factoriser votre code afin de le rendre mieux structuré et plus lisible.

Cette feuille d'exercices est divisée en trois sections. La première portant sur des notions de bases en POO, la deuxième sur les classes abstraites et la dernière sur les interfaces.

Les exercices de cette séance seront réalisés en Java. Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier `code`.

1 Consolidation - Programmation Orientée Objet

Question 1: (*10 minutes*) Encapsulation - Java
L'encapsulation sert à cacher les détails d'implémentation. Elle sert uniquement à montrer que les informations essentielles aux utilisateurs.

En Java, il est recommandé de déclarer les attributs comme étant `private` et de mettre à disposition des utilisateurs des méthodes publiques d'accès afin qu'ils puissent accéder ou modifier la valeur des attributs privés.

Le nom des méthodes d'accès commence par `get` ou `set`, suivi du nom de l'attribut dont la première lettre est mise en majuscule (Java Naming convention).

Le mot-clé `this` en Java est une variable de référence qui fait référence à l'objet actuel. Il est utilisé dans une méthode d'instance ou un constructeur pour accéder aux membres de l'objet actuel, tels que les variables d'instance, les méthodes et les constructeurs.

1. Dans votre IDE, créez une classe `Person`,
2. Ajoutez-y un attribut privé `name`,
3. Créez un getter et un setter pour l'attribut `name` en suivant la convention de nommage des méthodes.
4. Dans votre `main`, créez une instance de `Person`,
5. En utilisant le setter défini précédemment, donnez un nom (`name`) à votre instance.
6. Affichez le nom de l'instance en utilisant le getter de l'attribut `name`.

Question 2: (*10 minutes*) Héritage - Java
Comme vous le savez, il est possible que des classes-filles héritent des attributs ou méthodes de classes-mères. En Java, il faut utiliser le mot-clé `extends` lorsqu'on définit une classe-fille. Ainsi, l'héritage permet la réutilisation des attributs et méthodes d'une classe existante.

1. Créez une classe-mère `Vehicle` ayant un attribut protégé (`protected`) de type chaîne de caractères appelé `brand`.
2. Créez un constructeur pour la classe `Vehicle` et assignez la valeur passée comme argument à l'attribut `brand`.
3. Définissez une méthode `honk` qui affiche `"Tuut, tuut!"`
4. Créez une classe-fille `Car` qui hérite de `Vehicle` et ayant pour attribut `modelName` avec valeur par défaut `"Mustang"`.

Conseil

Dans le constructeur de la classe-fille, n'oubliez pas de faire appel au constructeur de la classe-mère en utilisant le mot-clé `super`.

Question 3: (*30 minutes*) [Java] Redéfintion des méthodes d'égalité et d'identité
En Java, créez une classe `Fraction` ayant pour attributs privés un numérateur et un dénominateur.

> **Conseil**
>
> Vous pouvez vous inspirer des solutions d'exercices de la semaine 8.

Assignez des valeurs à vos attributs dans le constructeur que vous définirez. Dans le corps de votre classe, effectuez les opérations suivantes :

— Définir une méthode nommée `gcd` qui prend comme arguments les deux attributs définis précédemment et qui retourne leur plus grand diviseur commun. Pour cela, vous devez implémenter l'algorithme d'Euclide de façon récursive.

> **Conseil**
>
> L'algorithme d'Euclide fonctionne comme suit :
>
> — Si a = 0 alors gcd(a, b) = b.
>
> — Si b = 0 alors gcd(a, b) = a.
>
> — Si aucune des conditions ci-dessus n'est remplie, faire appel à `gcd` de façon récursive avec pour premier argument `b` et pour deuxième `a%b`.

— Définir une méthode `simplify` qui sera appelée dans le constructeur après l'initialisation des attributs.

— Redéfinir la méthode `toString` pour produire une représentation textuelle des instances de `Fraction`.

— Redéfinir la méthode `equals` pour tester l'égalité entre deux fractions. Cette méthode prendra comme argument un objet `other` de type `Object`.

> **Conseil**
>
> En Java, par défaut toutes les classes héritent d'une classe-mère nommée `Object`. Vous puvez utiliser le même test d'égalité que vous avez fait pour les exercises de la semaine 7.

— Pour tester l'égalité dans la méthode `equals` :

— Vérifier si `this` et `other` sont le même objet. Si oui, renvoyer `true`.

— Vérifier si `other` est `null` ou si `this` et `other` sont de types différents. Si oui, renvoyer `false`.

— Sinon, vérifier si les deux fractions sont numériquement égales (même si leurs numérateurs et dénominateurs sont différents) et renvoyer le résultat de cette comparaison.

> **Conseil**
>
> En Java, pour vérifier que deux objets sont de même type, vous pouvez accéder à la classe de chacun des éléments en utilisant la méthode `.getClass()` et les comparer. Une fois que vous avez vérifé le type de l'objet `other` passé comme argument, le convertir en `Fraction` (utiliser type casting de la semaine 3).

— Dans le `main`, exécuter le code suivant (disponible sur Moodle) :

```
1 public class Question3 {
2     public static void main(String[] args) {
3         Fraction f1 = new Fraction(8, 32);
4         Fraction f2 = new Fraction(1, 4);
5         System.out.println(f1 == f2);
6         System.out.println(f1.equals(f2));
7     }
8 }
```

.

— Pourquoi les lignes 5 et 6 renvoient-elles des résultats différents ?

2 Classes abstraites

Question 4: (*15 minutes*) Création d'une classe abstraite
Une classe abstraite est une classe dont l'implémentation n'est pas complète et qui n'est pas instanciable. Elle est déclarée en utilisant le mot-clé `abstract`. Elle peut inclure des méthodes abstraites ou non. Bien que ne pouvant être instanciées, les classes abstraites servent de base à des sous-classes qui en sont dérivées. Lorsqu'une sous-classe est dérivée d'une classe abstraite, elle complète généralement l'implémentation de toutes les méthodes abstraites de la classe-mère. Si ce n'est pas le cas, la sous-classe doit également être déclarée comme abstraite.

```
1 // Exemple de classe abstraite
2 public abstract class Animal {
3     protected int speed;
4
5     // Déclaration d une méthode abstraite
6     abstract void run();
7 }
```

— Créez une classe abstraite appelée `Item` avec quatre attributs d'instance et un attribut de classe, qui sont les suivants :

```
1 private static int count = 0; // Attribut de classe
2 private String name;
3 private int id;
4 private double price;
5 private List<String> ingredients;
```

> **Conseil**
>
> Les attributs d'instance sont créés lors de l'instanciation d'un objet (à l'aide du mot-clé `new`) et détruits lors de la destruction de l'objet. Les attributs de classes (attributs statiques), quant à eux, sont créés lors de l'exécution du programme et détruits lors de l'arrêt du programme. En Java, les attributs de classes sont accessibles en utilisant le nom de la classe soit : `ClassName.attributeName`.

— Créez un constructeur pour initialiser les attributs `name, price, ingredients et id`. L'attribut `id` incrémentera à chaque instanciation de la classe.

> **Conseil**
>
> Pensez à utiliser `count` pour initialiser la valeur d'`id`. Ainsi, dans le constructeur, `id` sera égal à `++count`.

— Implémentez les `getters` des attributs `id, name, price et ingredients`.

— Implémentez la méthode `equals(Object o)`.

— Redéfinissez `toString()` pour renvoyer une chaîne de quatre lignes au format suivant, en remplaçant les valeurs entre accolades par celles de l'objet :

```
ID: {id}
Name: {name}
Price: {price} CHF
List of ingredients: {ingredients}
```

La liste des ingrédients est entourée de crochets et ses éléments sont séparés par une virgule suivie d'une espace, par exemple `[rice, tomato, onion]`.

> **Conseil**
>
> La méthode `equals` permet de comparer deux objets. Elle prend en paramètre un objet de type `Object` et renvoie `true` si l'objet actuel est égal à l'objet passé en paramètre.

Question 5: (*15 minutes*) Classe abstraite et types d'attributs

- Implémentez une classe abstraite `Figure` contenant deux attributs protégés de type `float` : `largeur` et `longueur`, un constructeur qui initialise ces deux attributs, et deux méthodes abstraites : `getAire`() et `getPerimetre`() qui renvoient un `float`.
- Créez deux classes `Carre` et `Rectangle` qui héritent de la classe `Figure`. Le constructeur de `Rectangle` prend une largeur et une longueur, celui de `Carre` ne prend que la longueur du côté. À l'intérieur de ces classes, implémentez les méthodes `getAire()` et `getPerimetre()`.
- Dans le `main` d'une classe `FigureMain`, créez un carré et un rectangle et affichez le périmètre du carré et l'aire du rectangle.

Conseil

Un attribut protégé est accessible aussi bien dans la classe-mère que dans la classe-fille. On utilise le mot-clé `protected` pour rendre des attributs protégés.
Pour rappel, pour créer une classe fille, on utilise le mot-clé `extends`. Par exemple : `public class Carre extends Figure`.

3 Interfaces

Question 6: (*15 minutes*) Interface et héritage (Liée à la question 4)

En Java, une interface se déclare comme suit :

```
1  public interface IMakeSound {
2      double MY_DECIBEL_VALUE = 75;
3
4      void makeSound();
5  }
```

Pour rappel : dans une interface Java, les variables sont toujours `public static final`, et les méthodes sont public et abstract par défaut.
Les méthodes déclarées dans une interface doivent être implémentées dans des sous-classes :

```
1
2  public class Cat extends Animal implements IMakeSound {
3      public void makeSound() {
4          System.out.println("I meow at" + MY_DECIBEL_VALUE + "decibel.");
5      }
6
7      // Implémentation d une méthode abstraite
8      void run() {
9          this.speed += 10;
10     }
11 }
```

- Créez une interface `Edible` contenant une méthode `eatMe` qui ne retourne aucune valeur.
- Créez une interface `Drinkable` contenant une méthode `drinkMe` qui ne retourne aucune valeur.
- Créez une classe `Food` qui hérite la classe `Item` (définie à la question 4) et qui implémente l'interface `Edible`. Ecrire le constructeur de `Food` et la méthode `eatMe` (dans la classe `Food`).

Conseil

Vous pouvez reprendre la classe `Item` de la question 4.
Dans la méthode `eatMe()`, vous pouvez simplement afficher un message en utilisant un `println`.

Certains aliments ne sont pas seulement `Edible` (mangeable) mais aussi `Drinkable` (buvable) comme les soupes par exemple.

— Créez une classe `Soup` qui hérite de `Food` et implémente l'interface `Drinkable`. Ensuite, écrivez à la fois un constructeur pour `Soup` ainsi que la méthode `drinkMe` (dans la classe `Soup`).

Vous pouvez ensuite tester les méthodes `eatMe()` et `drinkMe()` à l'aide des lignes suivantes, qui créent des instances de `Soup` et `Food`.

```
1 Soup s1 = new Soup("Kizili soup", 7.7, new ArrayList<String>(List.of("bulgur",
      "meat", "tomato")));
2 Food f = new Food("Stuffed peppers", 12, new ArrayList<String>(List.of("rice",
      "tomato", "onion")));
3 s1.eatMe();
4 s1.drinkMe();
5 f.eatMe();
```